



K.N. Toosi University of Technology

OFFLINE JAVA GUIDE

Chapter 0

Java Fundamentals

Everything you need to start writing Java — from program structure and data types to loops, conditionals, and the common pitfalls every beginner should know.

COURSE

Advanced Programming

SEMESTER

Spring 2026

DEPARTMENT

Computer Engineering

v1.0

Table of Contents

0.1	Anatomy of a Java Program	Structure & rules
0.2	Primitive Data Types	8 types, literals, naming
0.3	Operators	Arithmetic, logic, bitwise
0.4	Input & Output	Scanner, printf, formatting
0.5	Conditional Statements	if / switch / ternary
0.6	Loops	for, while, do-while, break
0.7	Type Casting	Widening, narrowing, parsing
0.8	Comments	Three styles
0.9	Common Mistakes & Tips	Pitfalls every beginner hits

Every Java program starts from a `main` method inside a class. Here is the simplest complete program:

```
1 public class HelloWorld {
2     public static void main(String[] args) {
3         System.out.println("Hello, World!");
4     }
5 }
```

JAVA

Line-by-Line Breakdown

LINE	CODE	MEANING
1	<code>public class HelloWorld</code>	Declares a public class named <code>HelloWorld</code>
2	<code>public static void main(String[] args)</code>	The entry point — JVM calls this method first
3	<code>System.out.println(...)</code>	Prints text to the console, then moves to a new line

Key Rules

QUICK REFERENCE — FILE & CLASS RULES

- › The file name **must exactly match** the public class name: `HelloWorld.java`
- › Each `.java` file can have **only one** `public` class
- › A file **can** contain multiple non-public classes
- › Java is **case-sensitive**: `helloworld` $\neq$ `HelloWorld`
- › Every statement ends with a **semicolon** `;`
- › Curly braces `{}` define a block of code — always use them, even for single-line blocks
- › Compilation: `javac HelloWorld.java` → produces `HelloWorld.class`
- › Execution: `java HelloWorld` (no `.class` extension)

NOTE

The keywords `public`, `static`, and `void` before `main` will make full sense when you study classes and methods. For now, treat `public static void main(String[] args)` as a fixed template you write at the start of every program.

Java has **8 primitive types** — they store plain values directly in memory (not references to objects). Choosing the right type saves memory and prevents bugs.

The 8 Primitives

TYPE	SIZE	RANGE	DEFAULT	EXAMPLE
byte	1 byte	-128 to 127	0	byte b = 100;
short	2 bytes	-32,768 to 32,767	0	short s = 30000;
int	4 bytes	±2.1 billion	0	int x = 42;
long	8 bytes	±9.2 × 10 ¹⁸	0L	long big = 9999999999L;
float	4 bytes	~6-7 decimal digits	0.0f	float f = 3.14f;
double	8 bytes	~15 decimal digits	0.0	double d = 3.14159;
char	2 bytes	0 to 65,535 (Unicode)	'\u0000'	char c = 'A';
boolean	1 bit*	true or false	false	boolean ok = true;

Declaration & Initialization

JAVA

```
1 // Declare first, assign later
2 int age;
3 age = 20;
4
5 // Declare and assign in one line
6 double gpa = 3.85;
7 char grade = 'A';
8 boolean isPassed = true;
9
10 // Multiple variables of the same type
11 int x = 1, y = 2, z = 3;
```

Literals & Suffixes

TYPE	LITERAL FORM	NOTES
Integer	<code>42</code>	Default is <code>int</code>
Long	<code>42L</code> or <code>42l</code>	Use uppercase <code>L</code> (lowercase looks like <code>1</code>)
Float	<code>3.14f</code> or <code>3.14F</code>	Must have <code>f</code> — bare <code>3.14</code> is <code>double</code>
Double	<code>3.14</code> or <code>3.14d</code>	Default floating-point type
Binary	<code>0b1010</code>	Base 2 — useful for bit manipulation
Hex	<code>0xFF</code> or <code>0xff</code>	Base 16 — common for colors, memory
Octal	<code>077</code>	Base 8 — rarely used
Char	<code>'A'</code> or <code>'\n'</code>	Single quotes, not double
String	<code>"Hello"</code>	Double quotes — not a primitive (it's an object)

You can use **underscores** in numeric literals to improve readability (Java 7+):

```
int billion = 1_000_000_000;
long creditCard = 6037_9975_1234_5678L;
```

TIP

Underscores can be placed anywhere **except** at the start, end, or adjacent to the decimal point.

Naming Conventions

ELEMENT	STYLE	EXAMPLE
Variables & methods	camelCase	<code>studentAge</code> , <code>calculateSum()</code>
Classes	PascalCase	<code>StudentRecord</code> , <code>BankAccount</code>
Constants	UPPER_SNAKE	<code>MAX_SIZE</code> , <code>PI</code>
Packages	all.lowercase	<code>com.example.project</code>

MISTAKE

`float f = 3.14;` → **Compile error!** Bare decimal literals are `double`. You must write `3.14f` or cast: `(float)3.14`.

0.3

Operators

Arithmetic Operators

OPERATOR	MEANING	EXAMPLE	RESULT
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 2	2 (int) / 2.5 (double)
%	Modulo (remainder)	5 % 2	1

WARNING

Integer division truncates — 7 / 2 is 3, not 3.5. At least one operand must be double for floating-point division: 7.0 / 2 → 3.5.

Relational Operators

OPERATOR	MEANING	EXAMPLE	RESULT
=	Equal to	5 == 5	true
≠	Not equal to	5 != 3	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
≥	Greater or equal	5 ≥ 5	true
≤	Less or equal	3 ≤ 5	true

Logical Operators

OPERATOR	MEANING	BEHAVIOR
&&	Logical AND	Short-circuits: if left is false, right is not evaluated
	Logical OR	Short-circuits: if left is true, right is not evaluated
!	Logical NOT	Inverts: !true → false

Assignment Operators

=

+=

-=

*=

/=

%=

x += 5; is shorthand for x = x + 5;. All compound assignment operators work the same way.

Increment & Decrement

JAVA

```
1  int i = 5;
2
3  // Prefix: increment FIRST, then use
4  int a = ++i;    // a = 6, i = 6
5
6  // Postfix: use FIRST, then increment
7  int b = i++;    // b = 6, i = 7
8
9  // Same logic applies to -- (decrement)
```

TIP

Avoid using `++` / `--` inside complex expressions — it makes code hard to read and is a common source of bugs. Use it as a standalone statement: `i++;`

Ternary Operator

JAVA

```
1  // condition ? valueIfTrue : valueIfFalse
2  int age = 20;
3  String status = (age ≥ 18) ? "Adult" : "Minor";
```

Operator Precedence (High → Low)

PRECEDENCE

OPERATORS

1 (highest)

`()` — parentheses

2

`++` `--` `!` (unary)

3

`*` `/` `%`

4

`+` `-`

5

`<` `>` `≤` `≥`

6

`=` `≠`

7

`&&`

8

`||`

9

`?:` (ternary)

10 (lowest)

`=` `+=` `-=` ... (assignment)

NOTE

When in doubt, use parentheses. `(a + b) * c` is always clearer than relying on precedence rules.

Console Output

METHOD	BEHAVIOR
<code>System.out.print(val)</code>	Prints value, cursor stays on the same line
<code>System.out.println(val)</code>	Prints value, then moves cursor to next line
<code>System.out.printf(format, args)</code>	Formatted output (like C's printf)

```
1 System.out.print("Name: ");
2 System.out.println("Ali");
3 System.out.printf("Age: %d, GPA: %.2f%n", 20, 3.85);
```

JAVA

Name: Ali Age: 20, GPA: 3.85

OUTPUT

printf Format Specifiers

SPECIFIER	TYPE	EXAMPLE	OUTPUT
<code>%d</code>	int, long	<code>printf("%d", 42)</code>	<code>42</code>
<code>%f</code>	double, float	<code>printf("%f", 3.14)</code>	<code>3.140000</code>
<code>%.2f</code>	double (2 decimals)	<code>printf("%.2f", 3.14)</code>	<code>3.14</code>
<code>%s</code>	String	<code>printf("%s", "Hi")</code>	<code>Hi</code>
<code>%c</code>	char	<code>printf("%c", 'A')</code>	<code>A</code>
<code>%b</code>	boolean	<code>printf("%b", true)</code>	<code>true</code>
<code>%n</code>	newline	<code>printf("Hi%n")</code>	<code>Hi\n</code>
<code>%10d</code>	int, right-padded to 10	<code>printf("%10d", 42)</code>	<code>42</code>
<code>%-10d</code>	int, left-padded to 10	<code>printf("%-10d", 42)</code>	<code>42</code>

TIP

Use `%n` instead of `\n` in `printf`. `%n` automatically uses the correct line separator for the operating system.

Console Input — Scanner

```

1  import java.util.Scanner;
2
3  public class ReadInput {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          System.out.print("Enter your name: ");
8          String name = sc.nextLine();
9
10         System.out.print("Enter your age: ");
11         int age = sc.nextInt();
12
13         System.out.printf("Hello %s, you are %d years old.\n", name, age);
14
15         sc.close(); // Always close the scanner
16     }
17 }

```

Scanner Methods

METHOD	READS	EXAMPLE INPUT
<code>nextLine()</code>	Entire line (String)	<code>Hello World</code>
<code>next()</code>	Single word (String)	<code>Hello</code> (stops at space)
<code>nextInt()</code>	Integer	<code>42</code>
<code>nextLong()</code>	Long integer	<code>9999999999</code>
<code>nextDouble()</code>	Decimal number	<code>3.14</code>
<code>nextBoolean()</code>	Boolean	<code>true</code>
<code>next().charAt(0)</code>	Single character	<code>A</code>

The `nextLine()` **trap**: After calling `nextInt()` , `nextDouble()` , etc., the newline character `\n` remains in the buffer. The next `nextLine()` call reads that leftover newline and returns an empty string.

Fix: Add an extra `sc.nextLine();` to consume the leftover newline:

```

int age = sc.nextInt();

sc.nextLine(); // consume leftover \n

String name = sc.nextLine(); // now works correctly

```

TRAP

if / else if / else

```
1  int score = 85;
2
3  if (score ≥ 90) {
4      System.out.println("Grade: A");
5  } else if (score ≥ 80) {
6      System.out.println("Grade: B");
7  } else if (score ≥ 70) {
8      System.out.println("Grade: C");
9  } else {
10     System.out.println("Grade: F");
11 }
```

JAVA

Grade: B

OUTPUT

Never put a semicolon after the condition:

```
if (x > 0); // ← this does nothing!
```

The empty statement is the body of the `if`. The real block below always executes.

MISTAKE

switch Statement

```
1  int day = 3;
2  switch (day) {
3      case 1:
4          System.out.println("Saturday");
5          break;
6      case 2:
7          System.out.println("Sunday");
8          break;
9      case 3:
10         System.out.println("Monday");
11         break;
12     default:
13         System.out.println("Other day");
14         break;
15 }
```

JAVA

Monday

OUTPUT

Don't forget `break;` ! Without it, execution **falls through** to the next case — a very common bug. Always end each case with `break;` unless fall-through is intentional.

WARNING

Enhanced switch (Java 14+)

```
1 String result = switch (day) {  
2     case 1, 2 → "Weekend";  
3     case 3, 4, 5, 6, 7 → "Weekday";  
4     default → "Invalid";  
5 };
```

JAVA

Uses arrow syntax (`→`) instead of colons. No fall-through risk. Can be used as an expression that returns a value.

Ternary as a Quick Conditional

```
1 int a = 10, b = 20;  
2 int max = (a > b) ? a : b; // max = 20
```

JAVA

QUICK REFERENCE — CONDITIONALS

- › `if` for branching logic, `switch` for matching a single value against many constants
- › Always use braces `{}` even for single-line bodies
- › Java does **not** support `switch` on `double` or `float`
- › Comparing floating-point with `=` is unreliable — use a tolerance: `Math.abs(a - b) < 1e-9`
- › For Strings, use `.equals()` not `=` (covered in Chapter 2)

for Loop

Best when you know the number of iterations in advance.

```
1 // Print numbers 1 to 5
2 for (int i = 1; i ≤ 5; i++) {
3     System.out.print(i + " ");
4 }
```

JAVA

1 2 3 4 5

OUTPUT

```
1 // Sum of first N natural numbers
2 int n = 100, sum = 0;
3 for (int i = 1; i ≤ n; i++) {
4     sum += i;
5 }
6 System.out.println("Sum = " + sum);
```

JAVA

Sum = 5050

OUTPUT

while Loop

Best when the number of iterations is unknown and depends on a condition.

```
1 // Keep reading input until 0 is entered
2 Scanner sc = new Scanner(System.in);
3 int num;
4 while (true) {
5     num = sc.nextInt();
6     if (num == 0) break;
7     System.out.println("You entered: " + num);
8 }
```

JAVA

do-while Loop

Guarantees the body executes **at least once** (condition is checked after the body).

```
1 int guess;
2 Scanner sc = new Scanner(System.in);
3 do {
4     System.out.print("Guess (1-10): ");
5     guess = sc.nextInt();
6 } while (guess < 1 || guess > 10);
```

JAVA

break & continue

KEYWORD

EFFECT

`break`

Immediately exits the **nearest** enclosing loop or switch

`continue`

Skips the rest of the current iteration, jumps to the next one

```
1 // Print odd numbers from 1 to 10
2 for (int i = 1; i ≤ 10; i++) {
3     if (i % 2 == 0) continue; // skip even numbers
4     System.out.print(i + " ");
5 }
```

JAVA

1 3 5 7 9

OUTPUT

Labeled break

Use a label to break out of a **specific** outer loop, not just the nearest one:

```
1 outer:
2 for (int i = 1; i ≤ 3; i++) {
3     for (int j = 1; j ≤ 3; j++) {
4         if (i == 2 && j == 2) break outer;
5         System.out.println(i + "," + j);
6     }
7 }
```

JAVA

1,1 1,2 1,3 2,1

OUTPUT

QUICK REFERENCE - COMMON LOOP PATTERNS

- › Sum: `sum += value;` inside loop
- › Count: `count++;` inside conditional
- › Find max: `if (val > max) max = val;`
- › Find min: `if (val < min) min = val;`
- › Flag-based search: `boolean found = false;` → set to `true`, then break
- › Reverse iteration: `for (int i = n-1; i ≥ 0; i--)`

MISTAKE

Off-by-one errors are the most common loop bug. Always double-check your boundary conditions: should it be `<` or `≤`? Is the starting index `0` or `1`? Trace the first and last iteration by hand.

Type casting converts a value from one type to another. Java has two kinds:

Widening Conversion (Implicit — Safe)

Converting from a smaller type to a larger type. No data loss. Java does this automatically.



Also: char → int → long → float → double

```
1  int myInt = 100;
2  double myDouble = myInt; // automatic: 100 → 100.0
3
4  char myChar = 'A';
5  int charCode = myChar;    // automatic: 'A' → 65
```

JAVA

Narrowing Conversion (Explicit — Risk of Data Loss)

Converting from a larger type to a smaller type. You **must** cast explicitly. Fractional parts are truncated, and overflow can occur.

```
1  double pi = 3.14159;
2  int intPi = (int) pi;    // 3 (fractional part lost)
3
4  int big = 130;
5  byte small = (byte) big; // -126 (overflow! byte max is 127)
```

JAVA

WARNING

Narrowing from `int` to `byte` / `short` doesn't just truncate — it **wraps around**. `(byte)130` becomes `-126`, not `127`. This is because Java uses two's complement representation.

Useful char ↔ int Tricks

```
1  char c = 'A';
2  int code = c;           // 65
3
4  char lower = (char)(c + 32); // 'a' (uppercase to lowercase)
5  char upper = (char)('z' - 32); // 'Z' (lowercase to uppercase)
6
7  // Check if a character is a digit
8  char ch = '7';
9  boolean isDigit = (ch ≥ '0') && (ch ≤ '9'); // true
```

JAVA

Parsing Strings to Numbers

Comments are ignored by the compiler. They exist for humans reading your code.

JAVA

```
1 // Single-line comment: explains one line or a short idea
2 int radius = 5; // can also be placed at end of a line
3
4 /*
5  * Multi-line comment:
6  * useful for longer explanations,
7  * blocking out code, or detailed notes
8  */
9
10 /**
11  * Javadoc comment:
12  * used to generate HTML documentation.
13  * Place this before classes, methods, and fields.
14  *
15  * @param args command-line arguments
16  * @return nothing
17  */
18 public static void main(String[] args) { }
```

TYPE	SYNTAX	WHEN TO USE
Single-line	// ...	Brief explanations, inline notes
Multi-line	/* ... */	Long descriptions, temporarily disabling code
Javadoc	/** ... */	Documenting classes, methods, and fields for API docs

TIP

Good comments explain **why**, not **what**. The code itself should show what is happening. Comments like `// increment i by 1` above `i++;` add noise, not value.

A collection of the most frequent errors beginners make — read these carefully to save hours of debugging.

1. Semicolon After if / for

```
1  // WRONG — empty body, block always runs
2  if (x > 0);
3  {
4      System.out.println("positive");
5  }
6
7  // CORRECT
8  if (x > 0) {
9      System.out.println("positive");
10 }
```

JAVA

2. Assignment `=` vs. Comparison `==`

```
1  // WRONG — assigns 5 to x, always truthy (non-zero)
2  if (x = 5) { }
3
4  // CORRECT
5  if (x == 5) { }
```

JAVA

NOTE

In Java, `if (x = 5)` is actually a **compile error** for primitive `int` (unlike C/C++). But `if (booleanVar = true)` will compile — stay alert.

3. Integer Division Surprise

```
1  int a = 5, b = 2;
2  System.out.println(a / b);          // 2 (not 2.5!)
3  System.out.println(a / (double)b); // 2.5
```

JAVA

4. Comparing Strings with `==`

```
1  String s1 = new String("Hello");
2  String s2 = new String("Hello");
3
4  System.out.println(s1 == s2);      // false (different objects)
5  System.out.println(s1.equals(s2)); // true (same content)
```

JAVA

`==` checks if two references point to the **same object**. `.equals()` checks if the **content** is the same. Always use `.equals()` for String comparison. (Detailed in Chapter 2)

5. Declaring Variables Inside Loops

JAVA

```
1 // If you need the value after the loop, declare OUTSIDE:
2 int sum = 0;
3 for (int i = 1; i ≤ 10; i++) {
4     sum += i;
5 }
6 System.out.println(sum); // OK - sum is in scope
7
8 // WRONG - result is not accessible after the loop
9 for (int i = 1; i ≤ 10; i++) {
10     int result = i * 2;
11 }
12 System.out.println(result); // Compile error: cannot find symbol
```

6. Unreachable Code

JAVA

```
1 System.out.println("Hello");
2 return;
3 System.out.println("World"); // Compile error: unreachable statement
```

7. Forgetting to Import

JAVA

```
1 // Without import, this won't compile:
2 Scanner sc = new Scanner(System.in); // error: cannot find symbol
3
4 // Add this at the top of your file:
5 import java.util.Scanner;
```

8. Using `next()` vs `nextLine()`

JAVA

```
1 Scanner sc = new Scanner(System.in);
2 // Input: "Hello World"
3 String a = sc.next(); // "Hello" (stops at whitespace)
4 String b = sc.nextLine(); // " World" (reads the rest of the line)
```

QUICK REFERENCE - DEBUGGING CHECKLIST

- › Read the **compile error message** carefully — it tells you the file, line number, and what's wrong
- › Check for **missing semicolons**, **mismatched braces**, **wrong case**
- › Verify **variable scope** — is the variable declared where you're using it?
- › Print intermediate values with `System.out.println()` to trace what's happening
- › When stuck, **simplify** — comment out code until the error disappears, then add back line by line
- › Common compile errors: `cannot find symbol` (typo or missing import), `class expected` (extra brace), `incompatible types` (wrong cast)

This document was compiled from the following resources. All content was adapted and restructured for educational use.

- [1] **Programiz — Java Programming**
java-programming tutorial sections on basics, data types, operators, control flow
<https://www.programiz.com/java-programming>
- [2] **GeeksforGeeks — Java**
Articles on type casting, Scanner class, operator precedence, common mistakes
<https://www.geeksforgeeks.org/java/>
- [3] **Oracle — Java Documentation**
Official Java Language Specification and API documentation
<https://docs.oracle.com/en/java/>
- [4] **Baeldung — Java Tutorials**
Practical guides on Java fundamentals and best practices
<https://www.baeldung.com/>
- [5] **W3Schools — Java**
Quick reference sections on syntax, data types, and operators
<https://www.w3schools.com/java/>

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com